# Manual de Instalación

Churn Dashboard — Predicción de Abandono de Clientes Olist

Equipo Microproyecto

Marzo 2026

## Contents

**1 Descripción general** **2**

**2 Requisitos previos** **2**

**3 Instalación paso a paso** **2**
- 3.1 Paso 1 — Actualizar el sistema . . . 2
- 3.2 Paso 2 — Instalar Node.js y npm . . . 2
- 3.3 Paso 3 — Instalar PM2 de forma global . . . 3
- 3.4 Paso 4 — Configurar arranque automático de PM2 . . . 3
- 3.5 Paso 5 — Instalar Python 3.12 + venv + pip . . . 3
- 3.6 Paso 6 — Clonar el repositorio . . . 3
- 3.7 Paso 7 — Ejecutar el script de despliegue automático . . . 3

**4 Verificación de la instalación** **4**
- 4.1 Estado del proceso PM2 . . . 4
- 4.2 Prueba del endpoint de predicción . . . 4
- 4.3 Acceso desde el navegador . . . 4

**5 Comandos de operación** **4**

**6 Resolución de problemas frecuentes** **5**
- 6.1 Error `ERR_HTTP_HEADERS_SENT` . . . 5
- 6.2 Error: `pandas`, `joblib` o `sklearn` no encontrado . . . 5
- 6.3 No se registran requests en `server.log` . . . 5
- 6.4 La página no carga desde el navegador . . . 6

**7 Seguridad y buenas prácticas** **6**

# 1 Descripción general

Este manual describe los pasos para instalar y poner en marcha el **Churn Dashboard** en una instancia **Ubuntu 22.04** (recomendado en AWS EC2). Al finalizar, el tablero estará accesible en el puerto `3000` del servidor y administrado por PM2.

### Componentes instalados

- **Frontend**: aplicación React/Vite compilada y servida por Express desde el directorio `dist/`.
- **Backend**: servidor Express (Node.js) en `server/index.js`, endpoint `POST /api/predict`.
- **Inferencia ML**: script Python `server/predict_joblib.py` con el modelo `best_model_logreg.joblib`.
- **Gestor de procesos**: PM2 mantiene el servicio activo tras reinicios.

### Repositorio

https://github.com/Jason-Morrill/Microproyecto_proyectogrado

# 2 Requisitos previos

| Componente | Versión mínima | Notas |
|---|---|---|
| Ubuntu / Debian | 22.04 LTS | Recomendado EC2 |
| Node.js | 18.x | LTS recomendado |
| npm | 9.x | Se instala con Node.js |
| Python | 3.12 | Con `venv` y `pip` |
| PM2 | 5.x | Instalado globalmente vía npm |
| Git | 2.x | Para clonar el repositorio |

**Puertos:** el Security Group (EC2) o firewall debe permitir tráfico entrante en el puerto `3000` (TCP) desde las IPs que accederán al tablero.

# 3 Instalación paso a paso

## 3.1 Paso 1 — Actualizar el sistema

```
sudo apt update && sudo apt upgrade -y
```

## 3.2 Paso 2 — Instalar Node.js y npm

```
sudo apt install -y nodejs npm
node -v    # debe mostrar v18.x o superior
npm -v
```

Si la versión de Node.js disponible en `apt` es antigua, instale la versión LTS mediante NodeSource:

```
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt install -y nodejs
node -v
```

### 3.3 Paso 3 — Instalar PM2 de forma global

```
sudo npm install -g pm2
pm2 -v
```

### 3.4 Paso 4 — Configurar arranque automático de PM2

```
pm2 startup systemd -u ubuntu --hp /home/ubuntu
```

El comando anterior imprimirá otro comando con `sudo`. Ejecútelo exactamente como aparezca en pantalla. A continuación:

```
pm2 save
```

### 3.5 Paso 5 — Instalar Python 3.12 + venv + pip

```
sudo apt install -y python3.12-venv python3-pip
python3 --version     # debe mostrar 3.12.x
python3 -m pip --version
```

### 3.6 Paso 6 — Clonar el repositorio

```
cd /home/ubuntu
git clone https://github.com/Jason-Morrill/Microproyecto_proyectogrado \
    Microproyecto_proyectogrado
cd /home/ubuntu/Microproyecto_proyectogrado
```

Si el repositorio ya existe, actualícelo:

```
cd /home/ubuntu/Microproyecto_proyectogrado
git pull --ff-only
```

### 3.7 Paso 7 — Ejecutar el script de despliegue automático

```
cd /home/ubuntu/Microproyecto_proyectogrado/Tablero_final
bash deploy_churn_dashboard.sh
```

El script realiza automáticamente:

1. Instalación de dependencias del frontend (`npm install`).
2. `npm run build` para compilar la aplicación Vite en `dist/`.
3. Instalación de dependencias del backend (`npm install` en `server/`).

4. Creación (o reparación) del entorno virtual Python en `server/.venv`.
5. Instalación de dependencias Python desde `server/requirements.txt`.
6. Inicio (o reinicio) del proceso `churn_dashboard` en PM2.
7. `pm2 save` para persistir el proceso.

Al finalizar correctamente verá en consola:

```
[PM2] Process churn_dashboard online
```

# 4 Verificación de la instalación

## 4.1 Estado del proceso PM2

```
pm2 status
```

El proceso `churn_dashboard` debe aparecer en estado **online**.

## 4.2 Prueba del endpoint de predicción

```
curl -sS -X POST http://127.0.0.1:3000/api/predict \
  -H "Content-Type: application/json" \
  -d '{
    "recency": 32,
    "frequency": 5,
    "monetary": 500,
    "avg_review_score": 4,
    "avg_delivery_days": 10,
    "avg_late_days": 0,
    "avg_num_items": 2,
    "avg_price_sum": 100,
    "avg_freight_sum": 15,
    "customer_state": "SP"
  }'
```

Respuesta esperada:

```
{"churnProbability": 0.1234}
```

## 4.3 Acceso desde el navegador

Abra un navegador y navegue a:

`http://<IP-PUBLICA-EC2>:3000`

Debe cargarse la interfaz del tablero con el formulario lateral de entrada de datos del cliente.

# 5 Comandos de operación

| Acción | Comando |
|---|---|
| Ver estado del proceso | `pm2 status` |
| Ver logs en tiempo real | `pm2 logs churn_dashboard --lines 100` |
| Ver logs de aplicación | `tail-f/home/ubuntu/Microproyecto_proyectogrado/Tablero_final/server/logs/server.log` |
| Reiniciar tras cambios | `pm2 restart churn_dashboard --update-env` |
| Eliminar y recrear proceso | `pm2 delete churn_dashboard`<br>`pm2 start .../server/index.js --name churn_dashboard`<br>`pm2 save` |

# 6 Resolución de problemas frecuentes

## 6.1 Error `ERR_HTTP_HEADERS_SENT`

**Causa:** doble respuesta HTTP generada en el servidor.

**Solución:**

```
cd /home/ubuntu/Microproyecto_proyectogrado
git pull --ff-only
pm2 restart churn_dashboard --update-env
pm2 logs churn_dashboard --lines 100
```

## 6.2 Error: `pandas`, `joblib` o `sklearn` no encontrado

**Causa:** el entorno virtual Python no tiene las dependencias instaladas o el proceso PM2 no usa el Python de `.venv`.

**Verificación:**

```
/home/ubuntu/Microproyecto_proyectogrado/Tablero_final/server/.venv/bin/
   python \
  -c "import pandas, joblib, sklearn; print('ok')"
```

**Solución si falla:**

```
cd /home/ubuntu/Microproyecto_proyectogrado/Tablero_final/server
source .venv/bin/activate
python -m pip install -r requirements.txt
deactivate
pm2 restart churn_dashboard --update-env
```

## 6.3 No se registran requests en `server.log`

1. Verifique la ruta exacta del log con `pm2 show churn_dashboard`.
2. Genere tráfico con `curl` (ver §4.2).
3. Confirme que el proceso usa el script correcto: `pm2 show churn_dashboard | grep script`.

### 6.4 La página no carga desde el navegador

1. Confirme que `pm2 status` muestra `churn_dashboard` en estado `online`.
2. Verifique que el puerto 3000 esté abierto en el Security Group de EC2 (Inbound rules: TCP 3000 desde su IP o `0.0.0.0/0`).
3. Pruebe localmente desde el propio servidor con `curl http://127.0.0.1:3000`.

## 7 Seguridad y buenas prácticas

- **Security Group EC2**: restrinja el puerto 3000 solo a las IPs que necesiten acceso. Para producción, coloque un balanceador de carga (ALB) con HTTPS y no exponga el puerto 3000 directamente.
- **Secretos**: no suba archivos `.env` con credenciales al repositorio. Use AWS Secrets Manager o variables de entorno del sistema.
- **Modelo**: versione el artefacto `best_model_logreg.joblib` en MLflow y no lo sobreescriba sin incrementar la versión.
- **Instancia EC2**: mantenga la instancia **detenida** (Stopped) cuando no esté en uso. **No la termine (Terminate)**, ya que se perdería la configuración instalada.